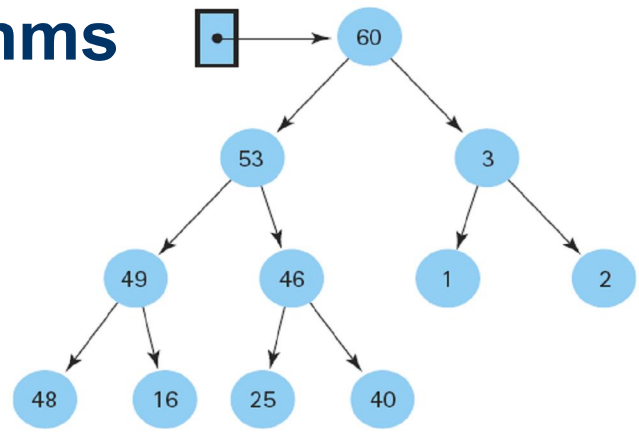
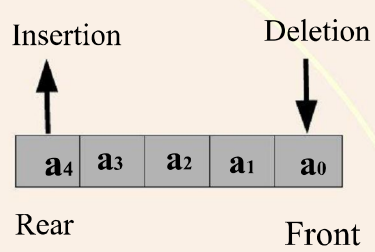
Data Structures & Algorithms



Queues

- $Q=(a_0,\ldots,a_{n-1})$
  - a_0 is the front of the queue
  - a_{n-11} is the rear of the queue
  - n is the number of elements

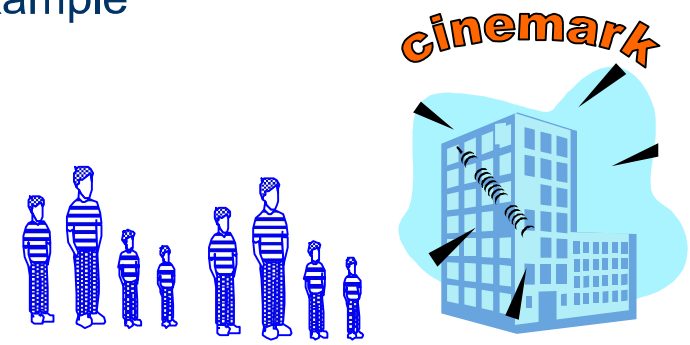
Example: queue of persons



- Linear data structure
- A queue is simply a waiting line that ***grows*** by adding elements to it's end and ***shrinks*** from it's front. Unlike a stack , a queue is a structure in which both the ends are used: one for adding new elements and one for removing them

Queue

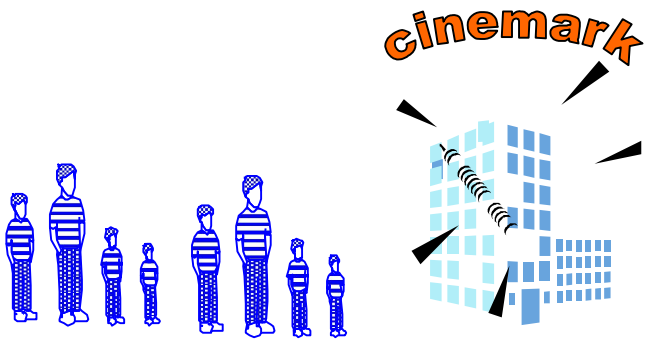
- Queue principle: FIRST IN FIRST OUT = FIFO
- It means: the first element inserted is the first one to be removed
- Example



- The first one in line is the first one to be served

Queue

- Insertions take place at the rear
- Deletions take place at the front

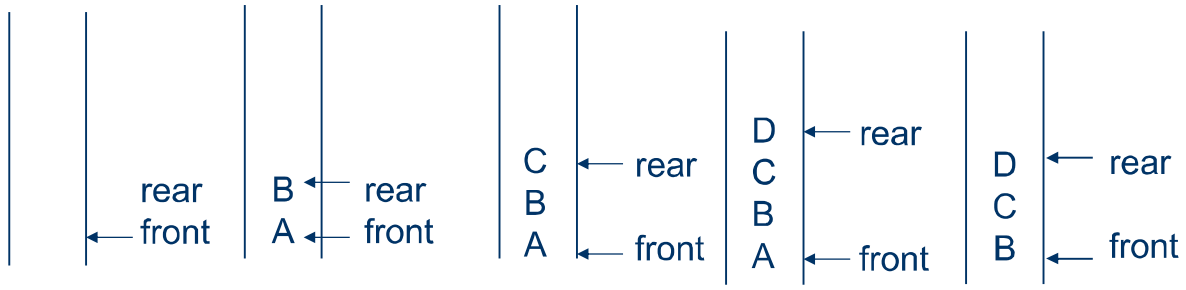


Queue Applications

- Real life examples
  - Waiting in line
- Applications related to Computer Science
- waiting lines in simulation problems
- scheduling processes by the operating system

  Job scheduling (e.g. Round-Robin algorithm for CPU allocation)
- access to shared resources

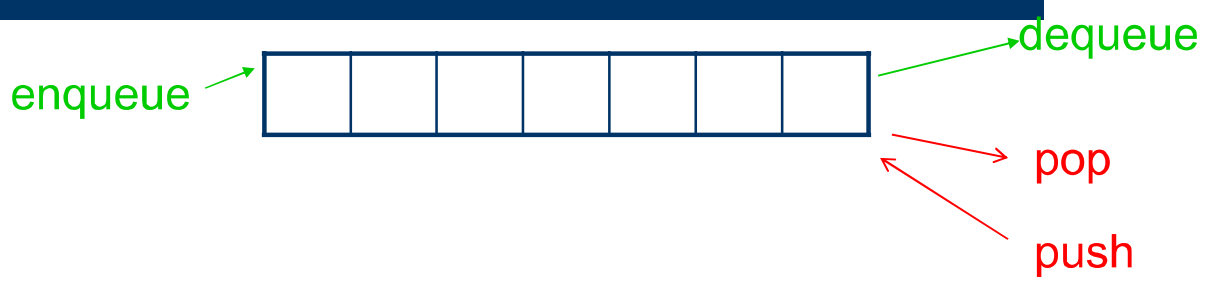
First In First Out



Operations of queues

enqueue(el) put the element el at the end of the queue;
dequeue() take the first element from the queue;
firstEl() return the first element in the queue without removing it;
clear() clear the queue;
isEmpty() check to see if the queue is empty;

Stack vs. queue



stack – a LIFO structure (last in, first out)
queue – a FIFO structure (first in, first out)

Queue implementation in c :

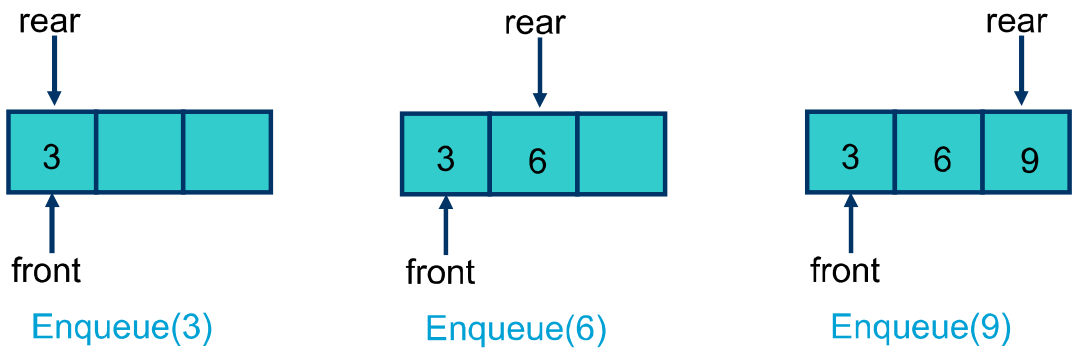
- Queue structures are usually implemented using arrays or linked lists.

Implementation of Queue

- Just as stacks can be implemented as arrays or linked lists, so with queues.
- Dynamic queues have the same advantages over static queues as dynamic stacks have over static stacks

Queue Implementation of Array

- As with the array-based stack implementation, the array is of fixed size
  - A queue of maximum N elements
- more complicated
  - Need to maintain track of both front and rear



ملاحظات :

- كيف تتم الاضافة ؟

تم الاضافة من نهاية الطابور (زيادة دليل الذيل واحد)

- كيف يتم الحذف ؟

من بداية الطابور (زيادة دليل الراس بواحد)

كيف نتأكد من ان الطابور فارغ ؟

الحالية الابتدائية للطابور ان يكون الراس = الذيل = -١

- كيف نتأكد ان الطابور ممتلئ ؟

اذا كان الذيل = الحجم – ١

- ماذا تعني الذيل = الراس = صفر او اي قيمة موجبة ؟

ان هناك عنصر واحد فقط

Enqueue Functions

```
#include<iostream>
using namespace std;
const int MAX=3;
void Enqueue(int queue[],int *front,int *rear ,int value)
{
    if (*rear < MAX-1)
    {
        *rear=*rear+1;
        queue[*rear]=value;
        if(*front==-1)
            *front=0;
        return;
    }
    else
        cout<<"queue is full\n";
}
```

Dequeue Functions

```
int Dequeue (int queue[],int *front,int *rear)
{
    int value;
    if (*front == -1)
        return -1;

    else
        if (*front<*rear)
    {

        value=queue[*front];
        *front=*front+1;
        return value;
    }
    else
        if (*front==*rear)
        {
            value=queue[*front];
            *front=*rear=-1;
            return value;
    }
}
```

Main Functions

```
void main(){
    int queue[MAX],front=-1,rear=-1,value,choice;
    char answer;

    cout<<"1. to Enqueue element\n"  <<"2. to Dequeue element\n";
    cout<<"want to try (y/n)\n";    cin>>answer;

    while(answer =='Y'||answer =='y')
    {
        cout<<"Enter your choise:\n"; cin>>choice;

        switch(choice)
        {
    case 1:cout<<"Enter the value to Enqueue:\n";cin>>value;
        Enqueue(queue,&front,&rear,value);          break;
    case 2:value=Dequeue(queue,&front,&rear);
        if(value!=-1)cout<<"the value Dqueue is "<<value<<endl;
        else cout<<"queue is empty\n";  break;
    default:cout<<"invalid choise\n";
        }
    cout<<"more ?(y/n)\n";  cin>>answer;
    }                                          }
```

END